

김찬민

Backend Engineer

근거로 선택하고, 검증으로 입증하는 백엔드 엔지니어입니다.

트레이드오프를 비교해 근거로 선택하고, 검증으로 입증합니다.

github.com/chanminkim-K

snoopy0813@naver.com

010-7664-4562

About Me

좋은 구조는 직감이 아니라 근거에서 시작한다고 생각합니다.
선택한 방향은 데이터와 테스트로 검증합니다.

Engineering Principles

- ✓ 문제를 먼저 정의합니다. problem first
- ✓ 트레이드오프를 비교합니다. compare → decide
- ✓ 선택은 데이터와 테스트로 검증합니다. decide → verify
- ✓ 실수에도 안전한 구조를 설계합니다. safe by design

Project Overview

두 프로젝트는 서로 다른 환경에서 진행했지만, 같은 방식으로 문제를 해결했습니다. 여러 대안의 트레이드 오프를 비교해 근거를 바탕으로 선택했고, 그 선택을 데이터와 테스트로 검증했습니다. 최신 실무 프로젝트 인 LMS는 현재의 개발 방식을, 수상작인 ERP는 이러한 사고방식이 이전부터 일관되게 이어져 왔음을 보여줍니다.



SaaS 멀티테넌트 LMS 플랫폼

MZC-LMS · 메가존클라우드 인턴십

1. Overview

기간	2025.11 – 2026.01 (3개월)
소속	메가존클라우드 · Ability Training Unit 팀 · 풀스택 개발 인턴
팀 규모	7명
기술 스택	Java · Spring Boot · Spring Data JPA · Spring Security · MySQL · React · TypeScript · Claude Code
한 줄 요약	하나의 플랫폼에서 여러 기업이 독립적으로 서비스를 사용할 수 있도록 멀티테넌트 구조를 설계한 SaaS LMS 프로젝트입니다. 데이터 격리·도메인 무결성·동시성·AI 협업 네 축을 설계·검증했습니다.

설계의 출발점

여러 기업이 하나의 플랫폼을 공유하면서도 데이터와 브랜드는 독립적으로 관리되어야 했습니다. 또한 운영 과정에서 발생하는 실수를 개발자의 주의가 아니라 구조적으로 방지하는 설계가 필요했습니다. 이러한 요구 사항이 데이터 격리, 도메인 설계, 성능 검증, AI 협업 체계라는 네 가지 설계 결정의 출발점이 되었습니다.

주요 기여

영역	주요 기여
① 멀티테넌트 데이터 격리	3단계 방어 구조 설계 (구현 협업)
② 도메인 라이프사이클	상태 전이 구조 설계 및 구현
③ 성능·동시성	락 전략 설계 및 구현
④ AI 협업 체계	Context Engineering 표준 설계

2. Problem

하나의 애플리케이션에서 여러 기업(테넌트)의 데이터를 함께 서비스하는 멀티테넌트 SaaS 구조였습니다. 이 과정에서 네 가지 핵심 문제가 있었습니다.

- 데이터 혼입 위험** — 한 테넌트 사용자가 다른 테넌트 데이터에 접근하면 안 되며, 쿼리 조건 누락 같은 인적 오류가 곧 유출로 이어질 수 있었습니다.
- 도메인 무결성** — 수강 차수 상태의 수동 조작이나 비정상 전이는 데이터 무결성을 깨뜨릴 수 있었습니다.
- 성능·동시성** — 데이터가 누적되면서 검색·집계 부하가 증가하고, 정원 관리에서는 동시 요청으로 Race Condition 이 발생할 위험이 있었습니다.
- AI 협업 파편화** — 7 명이 각자 다른 방식으로 LLM 을 활용하면서 코드 스타일 불일치, 컨벤션 위반, 반복적인 코드 리뷰 비용이 발생했습니다.

3. Design Decisions

① 멀티테넌트 데이터 격리

PROBLEM 다수 테넌트 데이터의 혼입을 방지해야 했습니다. 개발자가 tenant_id 조건을 누락하면 다른 테넌트 데이터가 조회될 위험이 있었습니다.

TRADE-OFF DB per Tenant(격리 수준 높음·비용 증가) / Schema per Tenant(논리적 분리·운영 복잡) / Row-level(리소스 효율·인적 오류 위험) 세 전략을 비교했습니다. 인턴 환경에서는 DB per Tenant가 비현실적이라고 판단해 Row-level을 선택했고, 인적 오류 가능성은 3단계 방어 구조로 보완했습니다.

DECISION 도메인 식별 → JWT 검증 → Hibernate Filter 자동 주입의 3단계 방어 체계를 설계했습니다.



EVIDENCE TenantResolver Filter가 요청의 서브도메인에서 tenant_id를 추출해 ThreadLocal에 저장합니다. 이후 Security Filter가 JWT의 tenant_id와 비교해 불일치 시 즉시 403을 반환합니다. 마지막으로 Hibernate Filter가 모든 Repository Query에 tenant_id 조건을 자동 주입하여 개발자가 조건을 누락해도 데이터 혼입이 발생하지 않도록 구현했습니다.

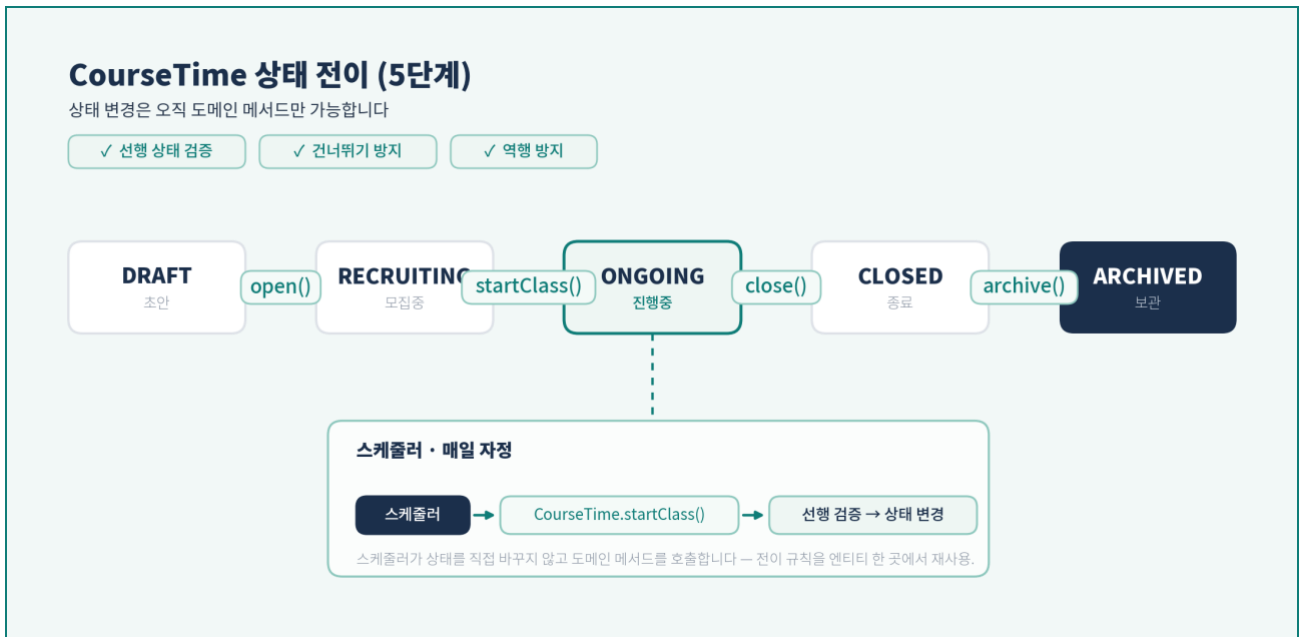
IMPACT 신규 테넌트 추가 시 코드 수정 없이 DB 레코드만 추가하면 자동 적용됩니다. 개발자가 tenant_id 조건을 누락해도 Hibernate Filter가 자동으로 적용되어 데이터 혼입을 구조적으로 방지합니다.

② 도메인 라이프사이클 (DDD)

PROBLEM 수강 차수(CourseTime) 상태의 수동 조작이나 비정상 전이는 데이터 무결성을 깨뜨릴 수 있었습니다.

DECISION 상태 전이를 엔티티 내부 도메인 메서드로 캡슐화했습니다. status는 private, 생성자는 protected, Setter를 제공하지 않았습니다. 상태 변경은 open() startClass() close() archive() 를 통해서만 가능하며, 각 메서드가 선행 상태를 검증해 건너뛰기와 역행을 구조적으로 방지합니다. 상태 전이 규칙은 엔티티 한 곳에서 관리하도록 설계했습니다.

상태는 DRAFT → RECRUITING → ONGOING → CLOSED → ARCHIVED의 5단계로 관리했습니다.



EVIDENCE ValidationRule enum으로 커스텀 제약 19종을 관리했습니다. 스케줄러는 상태를 직접 변경하지 않고 도메인 메서드를 호출해 전이 규칙을 재사용했으며, 상시 모집은 sentinel 값(9999-12-31)으로 처리했습니다.

[CODE-LMS-01] 상태 전이 캡슐화

```
// 선행 상태를 검증한 뒤에만 상태 변경을 허용
public void startClass() {
    if (this.status != CourseTimeStatus.RECRUITING) {
        throw new InvalidStatusTransitionException(
            this.status,
            CourseTimeStatus.ONGOING
        );
    }
    this.status = CourseTimeStatus.ONGOING;
}
```

상태 변경은 엔티티 내부 도메인 메서드를 통해서만 수행되며, 잘못된 전이는 예외로 차단됩니다.

IMPACT 어디서 상태를 변경하더라도 동일한 전이 규칙이 적용됩니다. 스케줄러까지 같은 도메인 메서드를 사용해 상태 변경을 자동화했습니다..

③ 성능 · 동시성

PROBLEM 데이터가 증가하면서 검색·집계 성능이 저하되고, 정원 관리에서는 동시 요청으로 Race Condition 이 발생할 수 있었습니다.

TRADE-OFF 무조건 비관적 락은 대기 비용이 크고, 무조건 낙관적 락은 경쟁 구간에서 재시도가 폭주합니다. 도메인의 충돌 특성에 따라 락 전략을 분리하는 것이 가장 합리적이라고 판단했습니다.

DECISION 고충돌 도메인(정원 관리·중복 배정)에는 비관적 락(PESSIMISTIC_WRITE)을 적용하고, 메타데이터와 같은 저충돌 도메인에는 @Version 기반 낙관적 락을 적용했습니다. 일부 도메인은 상황에 따라 두 전략을 병행했습니다.



EVIDENCE 검색은 다중 키워드 동적 필터와 서버사이드 페이지네이션으로 구현했습니다. Projection 8종, GROUP BY 집계 47개를 적용했고, Criteria API(Specification)와 Fetch Join으로 N+1을 방지했습니다. 전체 575개의 테스트와 멀티스레드 동시성 테스트 2종으로 락 전략을 검증했습니다.

[CODE-LMS-02] 충돌 특성에 따른 락 전략

```
// 고충돌 구간: 락 획득 후 모든 검증 수행
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<CourseTime> findWithLockById(Long id);

// 저충돌 구간: 커밋 시 버전 비교
@Version private Long version;
```

IMPACT 도메인 특성에 맞는 락 전략으로 데이터 정합성을 유지하면서 불필요한 락 비용을 줄였습니다. 또한 멀티스레드 테스트를 통해 선택한 전략이 의도대로 동작함을 검증했습니다.

④ AI 협업 체계

“AI를 단순히 사용하는 것이 아니라, 팀의 표준에 맞게 활용하는 체계를 설계했습니다.”

※ 이 결정은 코드 설계가 아니라 팀이 일하는 방식(System of Work)을 설계한 사례입니다.

PROBLEM 멀티테넌트 LMS 개발 과정에서 7명의 팀원이 LLM(Claude)을 활용했습니다. 그러나 팀원마다 프롬프트 작성 방식과 상세도가 달라, 같은 기능을 요청해도 결과 품질에 편차가 발생했습니다.

- 코드 스타일 불일치
- 프로젝트 컨벤션 위반
- 환각(Hallucination)으로 인한 잘못된 코드 생성
- 코드 리뷰 시 반복적인 스타일 수정 발생

문제의 원인은 프롬프트 작성 능력이 아니라, LLM이 프로젝트의 설계 기준과 컨벤션을 이해할 공통 맥락(Context)이 없었던 것이었습니다. Prompt Engineering은 개별 요청 최적화에는 효과적이었지만, 팀 단위에서 결과 품질을 일관되게 유지하기에는 한계가 있다고 판단했습니다.

TRADE-OFF Prompt Engineering은 요청마다 프롬프트를 최적화해 개별 결과의 품질을 높이는 데는 효과적입니다. 반면 Context Engineering은 프로젝트의 설계 기준·컨벤션·작업 절차를 먼저 구조화하여, 팀 전체가 동일한 기준으로 AI를 활용하도록 만듭니다. 팀 단위에서 품질의 일관성을 유지하기 위해 Context Engineering을 선택했습니다.

DECISION 프로젝트의 설계 기준과 작업 방식을 개인의 기억이 아니라 표준화된 컨텍스트가 전달하도록 설계했습니다. 이를 위해 CLAUDE.md 와 Context Engineering 문서를 중심으로 프로젝트 구조, 컨벤션, 작업 절차를 표준화하여 "누가 작업하더라도 같은 기준으로 개발하는" 협업 체계를 구축했습니다.

EVIDENCE 다음 내용을 프로젝트의 표준 문서로 정의하여 AI와 팀원이 동일한 기준을 공유하도록 했습니다.

- 프로젝트 구조 및 모듈 책임
- Controller-Service-Repository 계층 책임
- DTO · Entity 설계 규칙
- 예외 처리 및 응답 포맷
- Git · PR 컨벤션
- API 개발 · 버그 수정 · 코드 리뷰 · 테스트 생성을 위한 프롬프트 템플릿
- AI 작업 절차(계획 → 승인 → 작업 → 완료 보고)

```
# mzc-lp - AI 작업 가이드

> **핵심 원칙**: 이 문서만으로 대부분의 작업 시작 가능. 부족하면 부록 참조.
> **필수 작업 규칙**: 모든 작업은 **반드시 계획 먼저 제시** → 승인 → 순차 진행 → 완료 보고

---

## 프로젝트 개요

| 항목 | 내용 |
|-----|-----|
| **Backend** | Spring Boot 3.4.12, Java 21 |
| **Frontend** | React 19.x, TypeScript 5.x, Vite 7.x, TailwindCSS |
| **Database** | MySQL 8.0 |
| **Infra** | AWS (ECS, RDS, S3, CloudFront), Docker |
| **인증** | JWT (Access + Refresh Token) |

### 저장소 구조

> 폴리레포 (3개 저장소) → [POLY-REPO.md](./POLY-REPO.md)

---

## 핵심 규칙 (Must-Know)

### Backend
...


- Entity: Setter 금지 → 비즈니스 메서드 사용
- Service: @Transactional(readOnly=true) 클래스 레벨
- Controller: try-catch 금지 → GlobalExceptionHandler
- DTO: Java Record + from() 정적 팩토리
- Enum: @Enumerated(EnumType.STRING)


...

### Frontend
...


- any 타입 금지 → 명시적 타입 정의
- 서버 상태: React Query (useState는 UI 상태만)
- API: Axios Instance + handleApiError
- 컴포넌트: Props Destructuring + Early Return


...
```

```
## 컨벤션 로딩 (작업별 선택)

| 작업 | 필수 컨벤션 |
|-----|-----|
| 프로젝트 구조 | `01-PROJECT-STRUCTURE` |
| Git | `02-GIT-CONVENTIONS` |
| Controller | `03-CONTROLLER-CONVENTIONS` |
| Service | `04-SERVICE-CONVENTIONS` |
| Repository | `05-REPOSITORY-CONVENTIONS` |
| Entity | `06-ENTITY-CONVENTIONS` |
| DTO | `07-DTO-CONVENTIONS` |
| Exception | `08-EXCEPTION-CONVENTIONS` |
| React Core | `10-REACT-TYPESCRIPT-CORE` |
| React 구조 | `11-REACT-PROJECT-STRUCTURE` |
| Component | `12-REACT-COMPONENT-CONVENTIONS` |
| State 관리 | `13-REACT-STATE-MANAGEMENT` |
| API Service | `14-REACT-API-INTEGRATION` |
| Backend Test | `15-BACKEND-TEST-CONVENTIONS` |
| Frontend Test | `16-FRONTEND-TEST-CONVENTIONS` |
| Design/UI | `design/00-DESIGN-CONVENTIONS` |
| Docker | `17-DOCKER-CONVENTIONS` |
| Database | `18-DATABASE-CONVENTIONS` |
| AWS 배포 | `19-AWS-CONVENTIONS` |
| 보안 | `20-SECURITY-CONVENTIONS` |
| 성능 | `21-PERFORMANCE-CONVENTIONS` |
| 외부 API | `22-EXTERNAL-API-CONVENTIONS` |
| 멀티테넌시 | `23-MULTI-TENANCY` |
| Ignore 설정 | `24-IGNORE-CONVENTIONS` |
| 다국어(i18n) | `25-I18N-CONVENTIONS` |
| 코드 품질 분석 | `26-CODE-QUALITY-ANALYSIS` |
| AI 시대 개발자 역할 | `27-DEVELOPER-ROLE-AI-ERA` |

> 컨벤션 위치: `docs/conventions/`
```

```
## 작업 순서

**Backend CRUD**: Entity → Repository → DTO → Exception → Service → Controller → Test

**Frontend 페이지**: Types → API Service → React Query Hook → Component → Test

---

## 프로젝트 구조

...

backend/.../domain/
├─ user/           # controller, service, repository, entity, dto, exception
├─ course/
└─ enrollment/

global/           # config, exception, common

frontend/src/
├─ pages/          # 페이지 컴포넌트
├─ components/     # 재사용 컴포넌트 (common, layout)
├─ services/       # API 호출
├─ hooks/          # Custom Hooks (React Query 래핑)
├─ stores/         # 전역 상태 (필요시)
├─ types/          # TypeScript 타입
└─ utils/          # 유틸리티 함수
...
```

```
## AI 작업 규칙 (필수)

...
⚠️ 코드 작성 전 반드시 계획부터 제시할 것!
...

1. **계획 제시** → 작업 목록 테이블로 보여주기
2. **승인 대기** → 사용자 확인 후 진행
3. **순차 작업** → TodoWrite로 추적하며 진행
4. **완료 보고** → 결과 요약 제시

> 단순 질문/조회는 예외. 코드 생성/수정 작업은 무조건 계획 먼저.

---

## 워크플로우

**7단계**: 요구사항 → UX → 의존성 → 계획 → 리스크 → 구현 → 테스트

**MoSCoW 우선순위**:
- 🟡 Must - 필수 (릴리스 필수)
- 🟡 Should - 권장 (중요하나 필수 아님)
- 🟢 Could - 선택 (시간 허락시)
- ⚪ Won't - 제외 (이번 버전 제외)
```

- **도입 결과** — 코드 리뷰의 컨벤션 수정 건수가 리뷰당 약 5~7 건에서 1~2 건 수준으로 감소했습니다.
- **코드 스타일** — 작성자마다 달랐던 스타일이 팀 공통 스타일로 일관되게 유지되었습니다.

IMPACT 프로젝트의 설계 기준과 컨벤션이 사람마다 달라지지 않도록 문서 기반으로 일관되게 전달하는 협업 체계를 만들었습니다. 개인의 프롬프트 작성 역량보다 팀 전체가 동일한 기준으로 AI를 활용하도록 만드는 것에 집중했습니다.

LLM을 개발자를 대체하는 도구가 아니라, 팀의 설계 기준과 컨벤션을 일관되게 전달하는 협업 도구로 활용했습니다.

4. Validation

설계를 실제 코드로 검증한 결과입니다.

검증 항목	결과	근거
테스트	575개	@Test 실측 (동시성 테스트 2종 포함)
상태 전이	5단계	DRAFT→...→ARCHIVED, 엔티티 캡슐화
커스텀 제약	19종	ValidationRule enum
커스텀 Projection	8종	인터페이스 기반 조회
집계 쿼리	47개	GROUP BY 기반
락 전략	비관 4 · 낙관 15	충돌 특성별 분리
Fetch Join	18개	N+1 차단 (@EntityGraph 미사용)

5. Key Takeaway

문제를 개발자의 주위에 맡기기보다 구조로 해결하고, 그 설계를 데이터와 코드로 검증하는 개발 방식을 실무에서 경험한 프로젝트였습니다.

SaaS 기반 클라우드 ERP

OMZ ERP · 메가존클라우드 SaaS Java 개발자 양성과정

1. Overview

기간	2024.05 – 2024.12 (7개월)
역할	물류 도메인 개발 · 오픈소스 모니터링 구축 · Redis 캐싱 전략 설계
팀 규모	7명
기술 스택	Java · Spring Boot · JPA · React · MySQL · Redis · AWS · Docker · GitHub Actions · Prometheus · Grafana
수상	메가존클라우드 SaaS Java 개발자 양성과정 최우수상 (3개 팀 중 1위) 부산디지털혁신아카데미 해커톤 아이디어상 (2024.11)

주요 기여

영역	주요 기여
오픈소스 모니터링	Prometheus-Grafana 환경 설계 및 구축
Redis 캐싱	캐싱 전략 설계 및 구현
물류 도메인	도메인 전체 설계 및 구현
MSA	도메인 경계 정의 및 검증 (서비스 분리 팀)

설계의 출발점

운영 데이터를 기반으로 서비스 구조를 검증하고, 병목의 원인을 찾아 점진적으로 개선하는 아키텍처가 필요했습니다.

2. Decision Chain — Observe · Validate · Improve

이 프로젝트의 세 가지 의사결정은 연쇄적으로 이어집니다. 모니터링으로 운영 데이터를 확보했고, 이를 기반으로 MSA 전환 가능성을 검증했습니다. 구조를 이해한 뒤에는 도메인별 Redis 캐싱 전략을 적용하며 서비스를 점진적으로 개선했습니다.

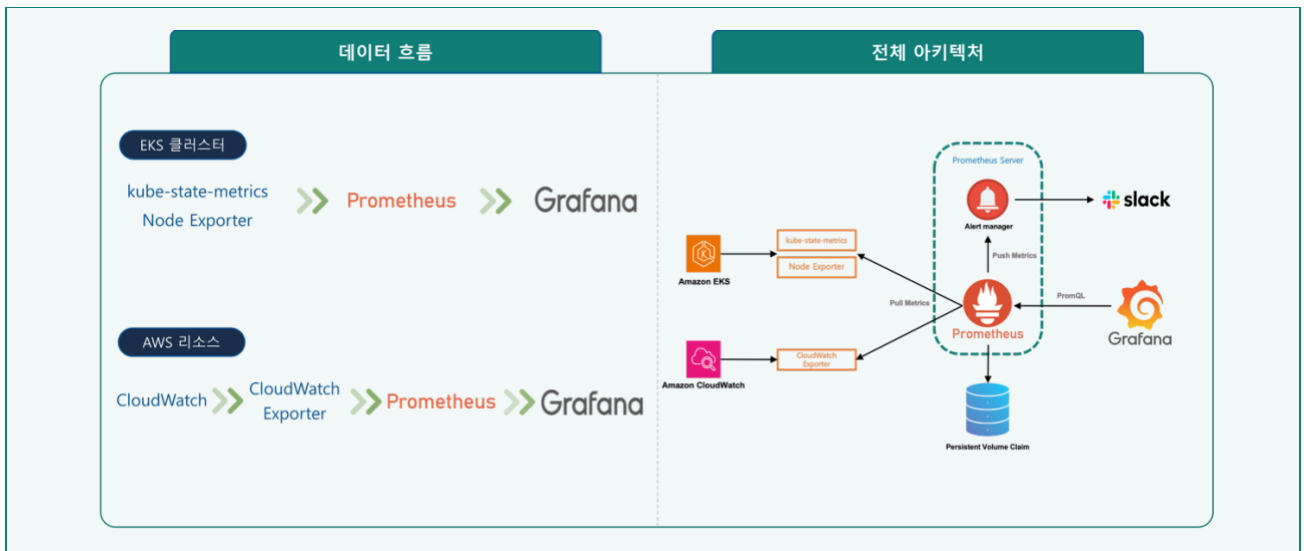


① Observe — 오픈소스 모니터링 환경 구축

PROBLEM 한정적인 비용을 고려해 medium EC2 워커 노드로 운영하면서 파드 자원 사용량과 애플리케이션 성능을 지속적으로 관측할 수단이 필요했습니다. 운영 데이터를 확보하지 못하면 이후 구조 개선의 근거를 만들 수 없었습니다.

TRADE-OFF 관리형 서비스(CloudWatch·Container Insights·AMP)는 구축이 간편하지만 대시보드 고정 요금과 사용량 기반 과금 부담이 크고, 클러스터 내부 지표를 유연하게 시각화하는 데 제약이 있었습니다. 오픈소스는 초기 구축 비용은 들지만 필요한 지표만 선택적으로 수집하고 자유롭게 확장할 수 있어, 비용 통제와 유연성을 고려해 선택했습니다.

DECISION Helm으로 Prometheus와 Grafana를 구축하고, 내부 리소스는 kube-state-metrics와 Node Exporter를 통해 수집했습니다. RDS·ALB 등 AWS 리소스는 CloudWatch Exporter를 추가해 하나의 Grafana 대시보드에서 통합 관측할 수 있도록 구성했습니다. 또한 AWS 문서를 참고해 필요한 Namespace와 Dimension만 values.yaml에 직접 매핑하여 핵심 지표만 선택적으로 스크랩하도록 설계했습니다.



CloudWatch Exporter를 활용한 AWS 지표 수집

- 세부 정보를 토대로 values.yaml 수정

세부 정보 편집

그래프로 표시된 모든 지표에 값을 적용하려면 다음을 클릭합니다.

Accountid [REDACTED]

네임스페이스 AWS/RDS

DBClusterIdentifier omz-erp-common-db

리전 ap-northeast-2

지표 이름 CPUUtilization

취소
업데이트

```

- aws_namespace: "AWS/RDS"
  aws_metric_name: "CPUUtilization"
  aws_dimensions: ["DBClusterIdentifier"]
  aws_statistics: ["Average"]
  
```

클러스터/노드 가용성 대시보드



IMPACT 인프라 자원과 애플리케이션 성능을 함께 관측할 수 있는 운영 데이터를 확보했고, 이후 서비스 분리 증 및 아키텍처 결정을 데이터 기반으로 판단할 수 있는 기반을 마련했습니다.

↓ Why Next? 운영 데이터는 확보했지만, 설계해 둔 서비스 분리(MSA)가 실제 운영에서도 타당한지는 아직 검증되지 않았습니다.

② Validate — 운영 데이터 기반 MSA 검증

PROBLEM 모놀리식 구조에서는 특정 도메인에 트래픽이 집중될 경우 전체 시스템의 성능 저하와 장애 전파가 발생할 수 있었습니다. 팀은 MSA 전환을 결정했지만, 도메인·DB 경계를 어떻게 나누는 것이 실제 운영 환경에서도 적절한지 검증할 객관적인 근거가 부족했습니다. 설계만으로 서비스를 분리하기에는 위험이 있었습니다.

VALIDATION Decision ①에서 구축한 Prometheus·Grafana 모니터링 환경을 활용해 운영 지표를 수집하고, K6(200 Virtual Users) 부하 테스트를 수행하여 애플리케이션과 데이터베이스의 응답 시간, 처리량, 자원 사용량 등을 함께 분석했습니다. 이를 통해 특정 기능의 트래픽이 어떤 도메인에 집중되는지 확인하고, 설계한 서비스 경계가 실제 트래픽 패턴과 일치하는지 검증했습니다.

TEAM RESULT 검증 결과를 바탕으로 도메인·DB 경계 기준의 4개 서비스 구조로 전환했고, 특정 도메인의 장애가 전체 시스템으로 확산되지 않는 구조(SPOF 제거)를 확보했습니다. 서비스의 물리적 분리는 팀이 함께 수행했습니다.

↓ Why Next? 서비스 분리로 장애 전파 위험은 줄었지만, 대용량 반복 조회에 따른 DB 병목은 여전히 남아 있었습니다. 마지막으로 조회 성능을 개선하면서도 데이터 정합성을 유지할 수 있는 캐시 전략이 필요했습니다.

③ Improve — 데이터 정합성 우선 Redis 캐싱

PROBLEM 1년 치 판매계획 조회와 전표 출력 등 대용량 반복 조회로 DB 병목이 발생했습니다. 재무·물류 데이터 특성상 캐시 일관성이 중요했으며, Spring Cache만으로는 요구한 동적 TTL 정책을 적용하기 어려웠습니다.

TRADE-OFF Cache-Aside는 구현이 단순하지만 캐시와 DB 간 불일치 위험이 있습니다. Read-Through/Write-Through는 쓰기 비용이 증가하지만 캐시와 DB의 일관성을 유지할 수 있습니다. ERP 특성상 데이터의 정합성을 우선해 Read-Through/Write-Through 전략을 선택했습니다.

DECISION 기존 EKS 환경에 Redis를 구축하고, Spring Cache의 한계를 보완하기 위해 동적 TTL과 엔티티 연관 관계를 자동 분석하는 캐시 관리 구조를 설계했습니다.

EVIDENCE RedisTemplate과 AOP를 활용해 캐시별 만료 시간을 제어하는 커스텀 어노테이션 (@RedisCacheable, @RedisCachePut)을 구현하여 Spring Cache의 동적 TTL 한계를 보완했습니다. 또한 JPA Metamodel을 활용해 엔티티의 연관관계를 자동 분석하고, 정·역방향 Dependency Graph를 생성하여 캐시 무효화 대상을 추적할 수 있는 기반 구조를 구축했습니다.

[CODE-ERP-01] Metamodel 기반 의존성 그래프 자동 구성

```
@PostConstruct
public void buildGraph() {

    Metamodel metamodel = entityManager.getMetamodel();

    // ① 모든 JPA 엔티티 자동 탐색
    for (EntityType<?> entity : metamodel.getEntities()) {

        String entityName = entity.getName();

        dependencyGraph.putIfAbsent(entityName, new HashSet<>());
        reverseDependencyGraph.putIfAbsent(entityName, new HashSet<>());

        entity.getAttributes().stream()

            // ② @OneToMany, @ManyToOne 등 연관관계만 추출
            .filter(Attribute::isAssociation)

            // ③ 정방향·역방향 Dependency Graph 동시 생성
            .forEach(attribute ->
                addDependency(
                    entityName,
                    resolveTargetEntity(attribute)
                )
            );
    }
}

/** 엔티티 연관관계를 정·역방향 그래프에 동시에 등록 */
private void addDependency(String source, String target) {

    dependencyGraph.get(source).add(target);

    reverseDependencyGraph
        .computeIfAbsent(target, key -> new HashSet<>())
        .add(source);
}

/** 연관 엔티티 타입 추출 (@OneToMany, @ManyToOne 등) */
private String resolveTargetEntity(Attribute<?, ?> attribute) {
    ...
}
```

JPA Metamodel을 활용해 모든 엔티티의 연관관계를 런타임에 자동 분석하고, 정방향·역방향 Dependency Graph를 동시에 구성했습니다. 새로운 엔티티가 추가되어도 수동 등록 없이 그래프가 자동 갱신되며, 캐시 의존성 관리의 기반 자료구조로 활용했습니다.

[CODE-ERP-02] 동적 TTL 캐싱 — RedisCacheAspect

```
// Read-Through Cache
@Around("@annotation(...redis.RedisCacheable)")
public Object cacheableProcess(ProceedingJoinPoint joinPoint) throws Throwable {

    RedisCacheable cacheable =
        findAnnotation(joinPoint, RedisCacheable.class);

    String cacheKey =
        generateKey(cacheable.cacheName(), joinPoint);

    // ① Cache Hit → Redis 데이터 반환
    if (Boolean.TRUE.equals(redisTemplate.hasKey(cacheKey))) {
        return redisTemplate.opsForValue().get(cacheKey);
    }

    // ② Cache Miss → 원본 메서드(DB) 실행
    Object result = joinPoint.proceed();

    // ③ 메서드별 TTL 정책으로 Redis 저장
    cacheWithTtl(
        cacheKey,
        result,
        cacheable.expireTime()
    );

    return result;
}

/** expireTime < 0 : 영구 저장 / 그 외 : TTL 적용 */
private void cacheWithTtl(
    String key,
    Object value,
    long expireSeconds
) {

    if (expireSeconds < 0) {
        redisTemplate.opsForValue().set(key, value);
    } else {
        redisTemplate.opsForValue().set(
            key,
            value,
            expireSeconds,
            TimeUnit.SECONDS
        );
    }
}
```

```
@RedisCacheable(
    cacheName = "salesPlan",
    expireTime = 300
)
public List<SalesPlanDto> findSalesPlans(...) {
    ...
}
```

```
public @interface RedisCacheable {

    String cacheName();

    long expireTime() default -1;
}
```

AOP 기반 Read-Through 캐싱을 구현하고, 커스텀 Annotation의 expireTime 속성으로 메서드 단위 Dynamic TTL을 지원했습니다.

캐시 정책을 비즈니스 코드와 분리하면서 데이터 특성에 따라 만료 시간을 유연하게 제어할 수 있도록 설계했습니다.

IMPACT 운영 데이터로 검증한 구조 위에 Redis 캐싱을 적용하여 대용량 조회의 DB 병목을 완화했고, 데이터 정합성과 성능을 함께 고려한 전체 아키텍처를 완성했습니다.

3. Validation

Observe → Validate → Improve의 세 가지 의사결정을 순차적으로 적용한 전체 아키텍처 개선 결과입니다.
특정 기술 하나의 효과가 아니라, 운영 데이터를 기반으로 구조를 개선한 최종 성과를 측정했습니다.

측정: K6 부하 테스트 (가상 사용자 200명, 5회 평균)

지표	Before	After	개선
평균 응답시간 (고부하 대용량 조회 API)	9.58s	1.84s	81% 단축
실패율	22.5%	1.8%	92% 감소
처리량	19.6 req/s	89.2 req/s	4.6배

※ Redis 단독 효과 : 물류·생산 DB 쓰기 CPU 77.7%→15.0% 감소.

4. Key Takeaway

운영 데이터는 단순한 모니터링 결과가 아니라, 설계의 타당성을 검증하고 다음 개선 방향을 결정하는 근거가 될 수 있음을 경험한 프로젝트였습니다.